Computer Science COMP-8610 --- Assignment #2

Deadline is: Tuesday, March 5th, 2026, by 11:59pm. Submit to BrightSpace. Also, the assignment must be submitted by groups of no more than 4 students.

Complete only Question #1, #2, #3, #4 below.

Question #1 Download the benchmark dataset, MNIST, from http://yann.lecun.com/exdb/mnist/. Implement multi-class classification for recognizing handwritten digits (also known as multiclass logistic regression ---this is simply a feedforward neural network with k output neurons, with one output neuron for each class, and each output neuron o_i returns the probability that the input data-point x_j is in class i) and try it on MNIST.

Comments: No need to implement almost anything in DL by your own (this is true in general); the software framework (ie, the DL platform) typically provides implementations for all the things discussed in class, such as the learning algorithms, the regularizations methods, the cross-validation methods, etc.

Use your favorite deep learning platform. A few candidates:

1. Marvin from http://marvin.is/
2. Caffe from http://caffe.berkeleyvision.org)
3. TensorFlow from https://www.tensorflow.org
4. Pylearn2 from http://deeplearning.net/software/pylearn2/
5. Theano, Torch, Lasagne, etc. See more platforms at http://deeplearning.net/software_links/.

Read the tutorial about your selected platform (eg, for TensorFlow: https://www.tensorflow.org/tutorials), try it on MNIST; note that the first few examples in the tutorials are typically on MNIST or other simple image datasets, so you can follow the steps.

Comments: MNIST is a standard dataset for machine learning and also deep learning. It's good to try it on one shallow neural network (with one output neuron; eg, for recognizing a character A from a not-A character) before trying it on a deep neural network with multiple outputs. Downloading the dataset from other places in preprocessed format is allowed, but practicing how to read the dataset prepares you for other new datasets you may be interested in (thus, please, read the MNIST website carefully).

1. Try the basic minibatch SGD as your learning algorithm. It is recommended to try different initializations, different batch sizes, and different learning rates, in order to get a sense about how to tune the hyperparameters (batch size, and, learning rate). Remember to create and use validation dataset! It will be very useful for you to read Chapter-11 of the textbook.

2. It is recommended to try, at least, another optimization method of your choice (SGD with momentum, RMSProp, RMSProp with momentum, AdaGrad, AdaDelta, or Adam) and compare its performances to those of the basic minibatch SGD on the MNIST dataset. Which methods you want to try and how many you want to try and compare is up to you and up to the amount of time you have left to complete the assignment. Remember, this is a research course. You may want to read Chapter-8 also.

Question #2 Consider the L^2-regularized multiclass logistic regression. That is, add to the logistic regression loss a regularization term that represents the L^2-norm of the parameters. More precisely, the regularization term is

$$\Omega(w, b) = \lambda \Sigma_i (\|w^i\|^2 + \|b^i\|^2)$$

where $\{w^i, b^i\}$ are all the parameters in the logistic regression, and $\lambda \in R$ is the regularization hyper-parameter. Typically, λ is about C/n where n is the number of data points and C is some constant in [0.01,100] (need to tune C). Run the regularized multiclass logistic regression on MNIST, using the basic minibatch SGD, and compare its results to those of the basic minibatch SGD with non-regularized loss, in Question #1.

Question #3 Try CNN (convolutional neural networks) on MNIST (or CIFAR or any dataset of your choice). Use the basic minibatch SGD as your learning algorithm, with and without regularizations (and compare them). Experiment with different regularization methods (dropout, L^2-norm, augmentation, early stopping, etc). You may need to read Chapter-7, Chapter-8 and Chapter-9.

Question #4 Build a three-layer feedforward neural network:

$$\boldsymbol{x} \rightarrow \boldsymbol{h}^1 \rightarrow \boldsymbol{h}^2 \rightarrow \boldsymbol{o}$$

The hidden layers $\boldsymbol{h}^1$ and $\boldsymbol{h}^2$ have width 500. Train the network for 250 epochs[1] and test the classification error. Do not use regularizations. Plot the cross-entropy loss on the batches and also plot the classification error on the validation data.

Comments: [1]Each epoch is a pass over the training data. Suppose you use batches of size b, and the training data set has n points, then an epoch consists of n/b batches. Note that you can divide the data set into batches, and then round robin over the batches. You can also randomly sample say 64 points for each batch. Either way is OK, and typically there is no performance difference between them. When these batches are randomly sampled, it is possible that some point are not in any of them, but we still call these batches a pass over the data.